CAMPUSFIX – SMART CAMPUS ISSUE TRACKER

A PROJECT REPORT

Submitted by

MOHAMED IRSHAADH

(240701323)

in partial fulfilment of the End Semester Examination for the course

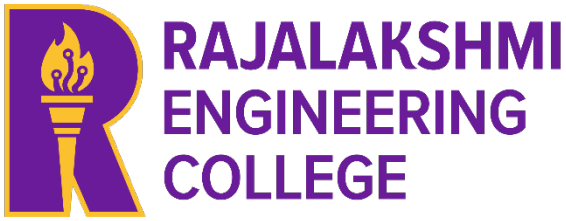
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "CampusFix – Smart Campus Issue Tracker" is the bonafide work of MOHAMED IRSHAADH (240701323), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. MANORANJINI
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. JINUSOPHIA
COURSE FACULTY

Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, MOHAMED IRSHAADH (240701323), hereby declare that the project report titled "CampusFix – Smart Campus Issue Tracker", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place:
Date:

SIGNATURE OF THE STUDENT
MOHAMED IRSHAADH
240701323

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Shankar Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinusophia, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

MOHAMED IRSHAADH

ABSTRACT

Campus problems such as Wi-Fi failures, faulty projectors, electrical faults and plumbing leaks are usually reported informally through conversations, phone calls and messages. As a result, a complaint has no shared record, the responsible department is guessed, reporters must follow up repeatedly, and administrators cannot see how many complaints are pending or which department is handling them.

CampusFix is a campus issue-tracking application developed on the Zoho Creator low-code platform. Exactly three forms hold the data: Department Directory, Report an Issue and Complaint Progress Updates. A lookup links each complaint to its department and another lookup links each progress update to a complaint. Four native form workflows implement the business rules in Deluge: a load workflow disables system-filled fields; an issue validation workflow assigns the active department for the selected category and initialises each new complaint as Submitted with a reporting timestamp; a progress validation workflow rejects a Resolved update without notes and records the updater and time; and a success workflow copies the new status to the linked complaint, so every action is kept as a separate history row. Three list reports and an Admin Dashboard page with four count cards, a status pie chart and a department column chart give administrators a consolidated view.

Nine functional test cases were executed in the live owner session on 9 November 2026 and all passed, covering routing for all five categories, initial values, status synchronisation, history retention, resolution-note validation and dashboard totals. The final demonstration holds five departments, five complaints and four progress updates. The project demonstrates relational low-code data modelling, event-based automation and native visualisation, while role-based access, notifications and external integrations are left as future work.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Complaint Tracking; Campus Maintenance*

PROJECT AT A GLANCE

Attribute	Details
Project Title	CampusFix – Smart Campus Issue Tracker
Application Domain	Campus administration / facility maintenance
Student Name & Register No.	MOHAMED IRSHAADH – 240701323
Zoho Creator Edition	Zoho Creator – trial plan (owner workspace)
Application Name (Link Name)	CampusFix (campusfix)
Published / Live URL	https://creatorapp.zoho.in/mohamedirshaadh.2024.cse1405/campusfix/
Demo Credentials (per role)	Owner account only – no credentials included in this report
Number of Forms / Subforms	3 / 0
Number of Reports (by type)	List: 3 Kanban: 0 Calendar: 0 Timeline: 0 Pivot/Chart: 0 (2 native page charts)
Number of Pages / Dashboards	1 (Admin Dashboard)
Number of Workflows	Form: 4 Schedule: 0 Approval: 0 Blueprint: 0
Deluge Functions / Custom Actions	4 workflow scripts / 0
Integrations	None
Roles & Profiles Defined	None separate – reporter and administrator paths demonstrated through the owner account
Environments Used	Single owner workspace (no Stage / Production promotion)
Date of Production Deployment	Not deployed to production – live demonstration verified 9 November 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 13
4.4 Process and Workflow Design 14
4.5 User Interface and Navigation Design 15
4.6 Access Control Design 16
IMPLEMENTATION 17
5.1 Application Setup 17
5.2 Forms and Data Capture 17

5.3 Reports 18
5.4 Pages and Dashboards 20
5.5 Deluge Scripting 21
5.6 Workflows and Automation 24
5.7 Integrations 25
5.8 User Management and Role-Based Access 26
5.9 AI and Analytics Features 26
5.10 Pro-Code Extension 26
5.11 Mobile and Portal Access 26
TESTING AND DEBUGGING 27
6.1 Testing Strategy 27
6.2 Test Cases 27
6.3 Debugging Deluge Scripts 29
6.4 Defect Log 30
6.5 User Acceptance Testing 30
DEPLOYMENT, SECURITY AND GOVERNANCE 31
7.1 Environments and Deployment Strategy 31
7.2 Performance Optimisation 31
7.3 Security Features 32
7.4 Governance and Administration 32
RESULTS AND DISCUSSION 34
8.1 Application Output 34
8.2 Achievement of Objectives 34
8.3 Comparison with the Existing System 35
8.4 Limitations of the Low-Code Approach 35
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 35
CONCLUSION AND FUTURE ENHANCEMENTS 37
9.1 Conclusion 37
9.2 Future Enhancements 37
9.3 Learning Outcomes and Career Relevance 37
REFERENCES 39
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 40
APPENDIX B – COURSE OUTCOME MAPPING 42
APPENDIX C – COMPLETE DELUGE SOURCE CODE 43
APPENDIX D – USER MANUAL 45
D.1 Accessing the Application 45
D.2 Reporting an Issue 45

D.3 Recording Progress (Administrator)..... 45
D.4 Reviewing and Maintaining..... 45
APPENDIX E – EVALUATION RUBRIC..... 46
APPENDIX F – VIVA-VOCE RECORD SHEET..... 48

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 5
Table 2.3: Informal Baseline versus CampusFix 7
Table 2.4: Category-to-Department Mapping 7
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 9
Table 3.4: Hardware and Software Requirements 9
Table 3.5: Use Case Description – Report an Issue 10
Table 3.6: Use Case Description – Record Progress Update 10
Table 4.1: Entities and Relationships 13
Table 4.2: Field Specification – Department Directory Form 13
Table 4.3: Field Specification – Report an Issue Form 13
Table 4.4: Field Specification – Complaint Progress Updates Form 14
Table 4.5: Role–Permission Matrix (Design – Not Yet Configured) 16
Table 5.1: Reports Implemented 19
Table 5.2: Verified Dashboard Counts 21
Table 5.3: Deluge Scripts Implemented 21
Table 5.4: Workflows Implemented 24
Table 6.1: Test Cases 27
Table 6.2: Defect Log 30
Table 6.3: User Acceptance Feedback 30
Table 7.1: Deployment Checklist 31
Table 7.2: Performance-Related Design Choices 32
Table 7.3: Security Controls and Remaining Boundaries 32
Table 8.1: Final Demonstration State (Fictional Records) 34
Table 8.2: Mapping of Objectives to Outcomes 34
Table 8.3: Existing System versus Proposed Application 35
Table A.1: Feature Coverage Checklist 40
Table B.1: Mapping of Course Outcomes to Report Chapters 42
Table E.1: Evaluation Rubric for the Individual Project 46
Table E.2: Performance Level Descriptors 47
Table F.1: Viva-Voce Questions and Responses 48
Table F.2: Marks Summary 48

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Application 10
Figure 4.1: System Architecture of the Application 12
Figure 4.2: Entity–Relationship Diagram 13
Figure 4.3: Workflow / Process Flow of a Complaint 15
Figure 4.4: Navigation Structure of CampusFix 15
Figure 5.1: Report an Issue Form before Submission 17
Figure 5.2: Complaint Progress Updates Form with Resolution Notes 18
Figure 5.3: Department Directory Entry Form with Active Default 18
Figure 5.4: Department Directory Report with Five Active Records 19
Figure 5.5: All Complaints Report with Assigned Departments and Statuses 20
Figure 5.6: Admin Dashboard with Count Cards, Status Chart and Department Workload Chart 20
Figure 5.7: Saved Category-Assignment and Initialisation Script in the Deluge Editor 22
Figure 5.8: Progress-Note Validation Script in the Deluge Editor 23
Figure 5.9: Linked-Complaint Status Synchronisation Script 24
Figure 5.10: Enabled Issue-Form Load and Validation Workflows 25
Figure 5.11: Enabled Progress Validation and Synchronisation Workflows 25
Figure 6.1: Complaint Update History with Four Retained Progress Records 28
Figure 6.2: Saved Resolution Update with Updater and Timestamp 28
Figure 6.3: Saved Wi-Fi Complaint Assigned to IT and Marked Resolved 29
Figure 6.4: Live Rejection of a Resolved Update without Notes 30

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AV	Audio-Visual
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
FR / NFR	Functional / Non-Functional Requirement
ID	Identifier
IT	Information Technology
ITIL	Information Technology Infrastructure Library
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
TC	Test Case
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 Background

Students and staff depend every day on working campus facilities: wireless network access in laboratories and classrooms, projectors and audio-visual equipment, lighting and power points, and plumbing in washrooms and common areas. When one of these fails, teaching and learning are interrupted, and the problem must reach the team that can fix it — IT support for the network, the audio-visual team for projectors, electrical maintenance for faults, and the facilities team for plumbing.

In practice these problems are reported informally. A student mentions a faulty projector to a faculty member, someone phones the office about a leaking tap, or a message is posted in a group chat. The details of the problem, its location and its follow-up remain spread across separate conversations. CampusFix gives the campus a single structured place to report issues, routes each issue to the right department automatically, and keeps a history of the actions taken until it is resolved. It is developed on Zoho Creator using forms, list reports, a dashboard page and Deluge workflow scripts.

1.2 Problem Statement

The campus has no single structured record that links an issue, its location, the responsible department and its progress history. The specific problems are:

- **No acknowledgement or record:** a reporter cannot confirm that a complaint was received, and the complaint can be forgotten.
- **Unclear responsibility:** whoever receives the complaint has to decide which department should handle it, and complaints are forwarded to the wrong team.
- **No visibility of pending work:** there is no way to see which complaints are still Submitted, In Progress or Resolved.
- **No history of actions:** the steps taken and the resolution details are not recorded, so a recurring problem cannot be traced.
- **No overview for administrators:** complaint counts by status and by department must be compiled manually, if at all.

These are the motivating design concerns; no campus-wide survey was carried out and no measured cost or time saving is claimed in this report.

1.3 Objectives

- To design three linked Zoho Creator forms that capture campus issues, department responsibility and progress updates.
- To assign every new complaint automatically to the active department responsible for its category using a Deluge validation workflow.
- To maintain a clear complaint lifecycle (Submitted → In Progress → Resolved) in which every status change is recorded as a separate progress row with notes, updater and time.
- To prevent a complaint from being marked Resolved without resolution notes.
- To give administrators list reports and an Admin Dashboard with status counts, a status chart and a department workload chart.
- To verify the application with documented functional test cases in the live Creator environment.

1.4 Scope of the Project

In scope. CampusFix is an individual academic mini project with exactly three primary forms (Department Directory, Report an Issue, Complaint Progress Updates), four native form workflows, three list reports and one dashboard page. It covers five issue categories — Wi-Fi, Projector, Electrical, Plumbing and Other — each mapped to one active department. All records used are fictional demonstration data in a live owner-account application.

Out of scope. Separate role-based permissions for reporters, technicians and administrators; e-mail or in-app notifications; external integrations and APIs; priority levels, deadlines and escalation; mobile and portal acceptance testing; production deployment across environments; and performance benchmarking. Uploading a photo is configured as an optional field but was not exercised in testing.

1.5 Justification for a Low-Code Approach

CampusFix is a small, data-centric workflow application: its value lies in structured records, relationships, a few business rules and summary views. Low-code platforms replace hand-written database schemas, CRUD screens, authentication and hosting with visual configuration and keep code for the business rules [1]. Zoho Creator's form builder provides typed fields and lookups between forms [2], form workflows attach

Deluge logic to load, validation and success events [3], and Deluge criteria fetch the records that match a category, status or identifier [4]. No separate front end, back end, database server or hosting was needed, and the complete application — forms, rules, reports and dashboard — was built and tested within the course timeline. Industry analysts report broad adoption of low-code platforms for exactly this kind of internal workflow application [5].

1.6 Organisation of the Report

Chapter 2 studies the existing system and related literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator with screenshots of the live application. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing informal process (Figure 2.1), a student or staff member who notices a problem tells a faculty member or office staff, or sends a message or makes a phone call. The receiver guesses which team is responsible and forwards the message. The team may or may not note the complaint; there is no shared record and no reference number. The reporter follows up repeatedly to learn whether anything has been done, and when the work is complete the closure is known only to the team. No history is kept, and nobody can say how many complaints are pending or which department receives the most.

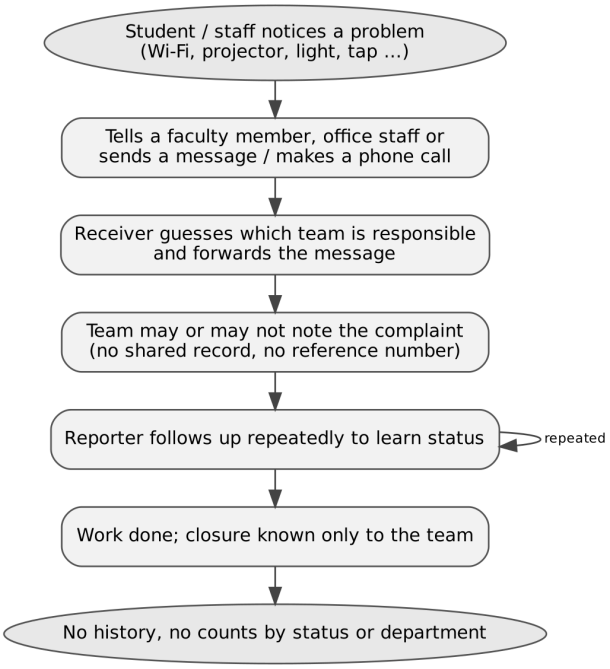


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Complaint details live in separate messages or notes and can be lost.
- Responsibility depends on the person who receives the complaint, so routing is inconsistent.
- The latest status of a complaint is unclear and requires manual follow-up.
- Actions taken and resolution details are not recorded, so there is no history.

- Administrators have no consolidated view of complaint status or department workload.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development effort	Database schema, REST API, front-end screens, authentication and hosting to be built	Forms, lookups, reports and page configured visually; four short Deluge scripts
Skills required	Front-end, back-end, SQL and deployment	Data modelling, Deluge scripting, platform configuration
Hosting & infrastructure	Server, database and SSL managed by the developer	Hosted by Zoho
Mobile access	Separate responsive UI or app	Available through the Creator mobile app (not tested in this project)
Security	Login and permissions coded manually	Zoho account login; roles and permissions available as configuration
Changing a mapping	Code change and redeploy	Edit a Department Directory record
Customisation limits	Unlimited	Layout and chart options constrained; standalone pivot charts need Zoho Analytics

2.4 Literature Survey

Table 2.2 summarises the sources reviewed. Research on low-code platforms [6], [7], [8] explains why the paradigm suits small business-process applications, while service-management practice [9] and an established ticketing product [10] describe how reported issues should be categorised, routed and tracked.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Sanchis et al., 2020 [6]	Low-code as enabler of digital transformation in manufacturing industry	Low-code lets domain users build process applications quickly.	Supports building a campus complaint tracker without a dedicated development team.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
2	Bock & Frank, 2021 [7]	Low-code platform	Characterises low-code platforms by their data, process and UI generators; notes flexibility limits and lock-in.	Informed which parts to configure and the limitations in Section 8.4.
3	Di Ruscio et al., 2022 [8]	Low-code development and model-driven engineering	Low-code is a practical form of model-driven engineering: models are executed directly.	Forms and workflows are treated as the executable design of CampusFix.
4	AXELOS, 2019 [9]	ITIL 4 – Incident management and service request practices	Issues should be logged, categorised, routed and tracked through defined states to closure.	Basis for category-based routing and the Submitted → In Progress → Resolved lifecycle.
5	ServiceNow [10]	Incident Management (commercial ITSM product)	Tickets carry category, assignment group, state and work notes.	Benchmark: CampusFix keeps the essential fields at far lower complexity.
6	Zoho Corporation [2], [3]	Zoho Creator Help – Form Builder, Form Workflows	Forms provide typed fields and lookups; workflows separate validation from post-save actions.	Directly used to design the forms and the four workflows.

2.5 Emerging Trends in Low-Code Development

Citizen development places small applications in the hands of the people who own the process; in CampusFix the category-to-department mapping is data in the Department Directory, so an administrator can change it without editing code. **Hyper-automation** combines rules and events into end-to-end processes; here, validation and success workflows connect issue registration, routing, history and status without manual hand-offs. **AI-assisted development**, such as Zia and AI app builders, is now built into platforms like Zoho Creator; the application's forms and scripts are documented and verified in this report, and AI assistance was used during application configuration, scripting, testing support and report preparation, with all outputs reviewed by the student. **Composable applications** expose data through APIs so that tools can be combined; CampusFix does not yet use integrations, but Creator's REST API would allow future connections such as notifications.

2.6 Proposed System

CampusFix stores every issue once in a structured form, determines the responsible department from a category mapping, and records each subsequent action as a related progress row. The current complaint status is a convenient summary, while the update report retains the full action history. Table 2.3 contrasts the proposed system with the informal baseline, and Table 2.4 lists the category mapping used in the demonstration.

Table 2.3: Informal Baseline versus CampusFix

Aspect	Informal Baseline	CampusFix
Complaint details	Messages or separate notes	One structured issue record
Responsibility	Manual forwarding	Active department assigned through a lookup
Progress	Latest message may be unclear	Current status plus linked update history
Administrative view	Records compiled manually	Count cards, charts and list reports

Table 2.4: Category-to-Department Mapping

Category	Department	Contact Person (fictional)
Wi-Fi	IT	Arun Kumar
Projector	AV Support	Priya Raman
Electrical	Electrical	Karthik S
Plumbing	Maintenance	Meena Devi
Other	General	Ravi Mohan

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Reporter (student / staff)	Person who notices a campus problem	Submit an issue with title, category, location and description; view complaint status
Administrator	Campus administration / maintenance coordinator	Maintain Department Directory; record progress updates; review reports and the Admin Dashboard
Department team (indirect)	IT, AV Support, Electrical, Maintenance, General	Perform the repair; their actions are recorded by the administrator as progress updates

Both the reporter and administrator paths were demonstrated through the authorised owner account; separate role-based permissions were not introduced in this version.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall capture an issue with title, category, location and description (photo optional).	Reporter	High
FR-02	The system shall maintain departments with the category handled, contact person, contact e-mail and active status, with unique department names.	Administrator	High
FR-03	On submission of a new issue, the system shall assign the active department for its category and set Status to Submitted and Reported At to the current time.	System	High
FR-04	The system shall reject an issue whose category has no active department.	System	Medium
FR-05	The system shall prevent direct editing of a complaint's status and require status changes through Complaint Progress Updates.	System	High
FR-06	The system shall record each progress update as a separate record linked to an existing complaint, with the updater and update time filled automatically.	Administrator	High
FR-07	The system shall reject a Resolved update whose notes are blank.	System	High

FR ID	Requirement	Role(s)	Priority
FR-08	After a valid update, the system shall set the linked complaint's Status to the new status.	System	High
FR-09	The system shall provide list reports of complaints, departments and update history, and an Admin Dashboard with status counts and charts by status and department.	Administrator	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Usability	Forms shall be readable, use consistent drop-down choices and show clear validation messages.
NFR-02	Data integrity	Lookups shall link records by record ID rather than by copied department names or issue titles.
NFR-03	Traceability	Every status change shall leave a separate progress record with notes, updater and time.
NFR-04	Maintainability	Category routing shall be changed by editing Department Directory records, not scripts; scripts shall be short and documented.
NFR-05	Availability	The application shall run on Zoho's hosted infrastructure; no numerical response-time or load guarantee was established for this prototype.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial plan, owner workspace)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not required (no integrations)
Other Zoho services	None
Client hardware	Any computer with an internet connection and a modern browser

3.5 Use Case Model

Figure 3.1 shows the use cases of the reporter and the administrator. Table 3.5 and Table 3.6 describe the two central use cases.

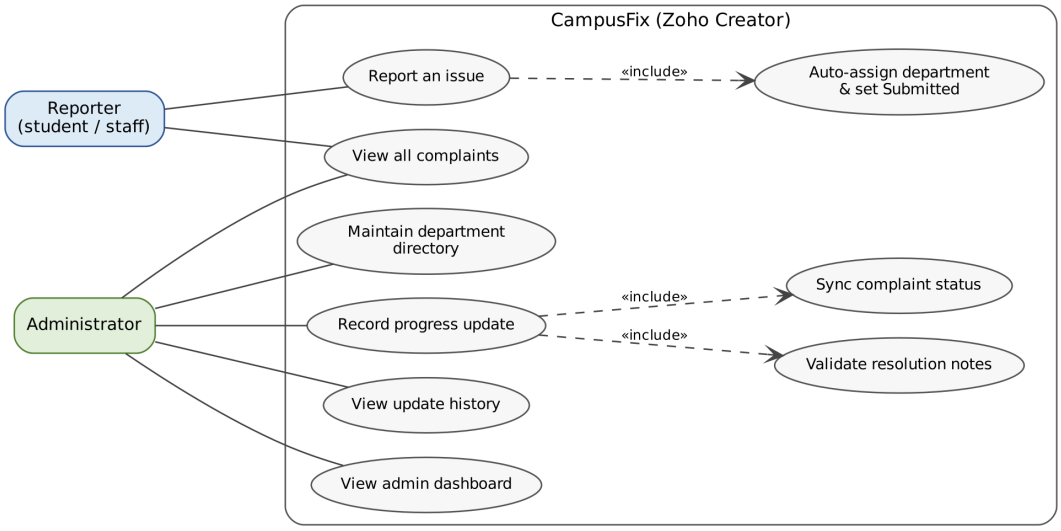


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Report an Issue

Use Case ID	UC-01
Use Case Name	Report an Issue
Actor(s)	Reporter
Pre-condition	An active Department Directory record exists for the category.
Main Flow	1. Open Report an Issue. 2. Enter title, category, location and description; optionally attach a photo. 3. Submit. 4. The system assigns the active department, sets Status to Submitted and Reported At to the current time.
Alternate Flow	4a. No active department for the category → alert shown and the submission is cancelled.
Post-condition	The complaint appears in All Complaints and in the dashboard counts.
Related FR	FR-01, FR-03, FR-04

Table 3.6: Use Case Description – Record Progress Update

Use Case ID	UC-02
Use Case Name	Record Progress Update
Actor(s)	Administrator
Pre-condition	The complaint exists in Report an Issue.

Main Flow	1. Open Complaint Progress Updates. 2. Select the complaint, choose New Status and enter notes. 3. Submit; the system fills Updated By and Update Date. 4. After saving, the linked complaint's Status is set to New Status.
Alternate Flow	2a. No complaint selected → alert, submission cancelled. 2b. New Status = Resolved with blank notes → alert, submission cancelled.
Post-condition	A new history row exists in Complaint Update History and the complaint shows the new status.
Related FR	FR-06, FR-07, FR-08

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

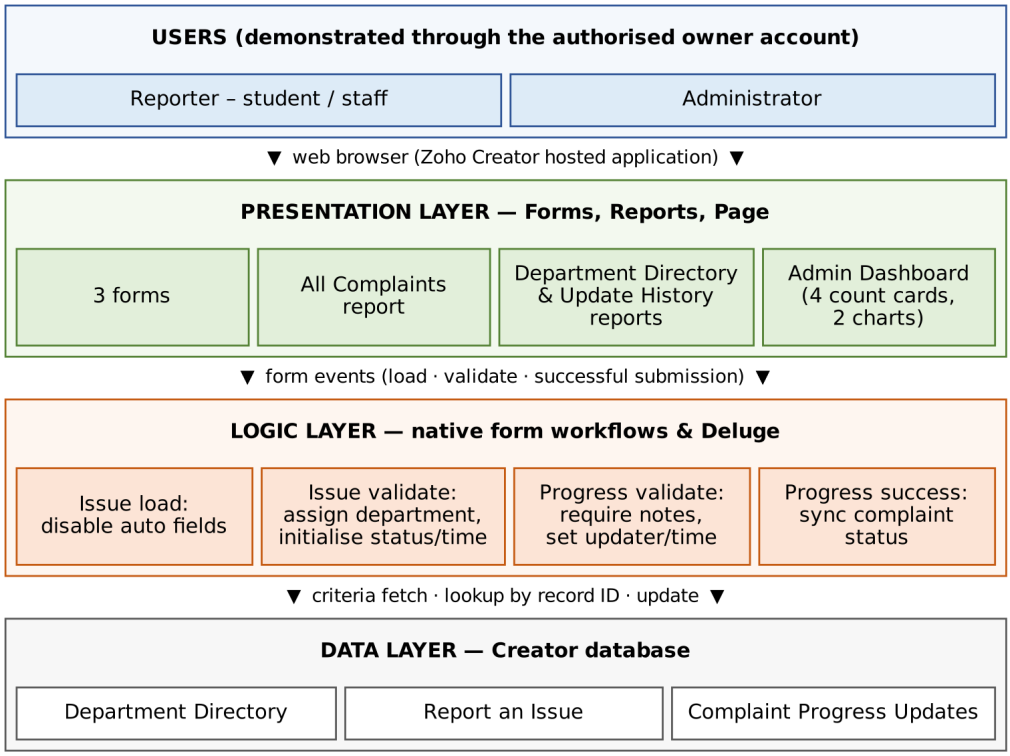


Figure 4.1: System Architecture of the Application

CampusFix follows the layered architecture in Figure 4.1. Users reach the hosted Creator application through a web browser. The **presentation layer** contains the three forms, three list reports and the Admin Dashboard page. The **logic layer** consists of four native form workflows written in Deluge, attached to the form-load, validation and successful-submission events. The **data layer** holds the three forms' records in the Creator database, connected by lookups. No external service is needed to route a complaint or update a related record.

4.2 Data Model

Figure 4.2 shows the entity–relationship model. One department can be assigned to many complaints, and one complaint can have many progress updates. Both relationships are lookups that store the related record's ID, which preserves identity even if a department name or issue title is later edited.

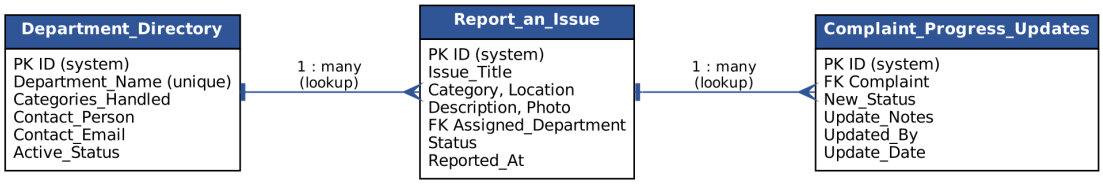


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Department_Directory	ID, Department_Name	Report_an_Issue	One-to-Many	Lookup Assigned_Department
Report_an_Issue	ID, Issue_Title	Complaint_Progress_Updates	One-to-Many	Lookup Complaint

4.3 Form Design

Table 4.2: Field Specification – Department Directory Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Department Name	Department_Name	Single Line	Yes	No duplicate values	e.g. IT
Categories Handled	Categories_Handled	Drop Down	Yes	Wi-Fi, Projector, Electrical, Plumbing, Other	Used for routing
Contact Person	Contact_Person	Single Line	No	—	Fictional names
Contact Email	Contact_Email	Email	No	Valid e-mail	*.demo@example.edu
Active Status	Active_Status	Drop Down	Yes	Default: Active	Only Active records are routed

Table 4.3: Field Specification – Report an Issue Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Issue Title	Issue_Title	Single Line	Yes	—	Short summary
Category	Category	Drop Down	Yes	Wi-Fi, Projector, Electrical, Plumbing, Other	
Location	Location	Drop Down	Yes	Block A, Block B, Library, Computer Lab, Auditorium, …	
Description	Description	Multi Line	Yes	—	

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Photo	Photo	File Upload	No	—	Optional; upload not tested
Assigned Department	Assigned_Department	Lookup (Department Directory)	—	Set by DS-02	Disabled on load (DS-01)
Status	Status	Drop Down	Yes	Submitted / In Progress / Resolved	Set by DS-02 and DS-04
Reported At	Reported_At	Date-Time	—	Set by DS-02	Disabled on load

Table 4.4: Field Specification – Complaint Progress Updates Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint	Complaint	Lookup (Report an Issue)	Yes	Checked by DS-03	Shows Issue Title
New Status	New_Status	Drop Down	Yes	In Progress / Resolved	
Update Notes	Update_Notes	Multi Line	For Resolved	Non-blank when Resolved (DS-03)	
Updated By	Updated_By	Single Line	—	zoho.loginuserid (DS-03)	
Update Date	Update_Date	Date-Time	—	zoho.currenttime (DS-03)	

4.4 Process and Workflow Design

Figure 4.3 shows the complaint workflow. A new complaint is assigned and timestamped during validation. Status changes are then recorded only through the progress form: a Resolved update needs non-blank notes, and a valid update is saved before its status is copied to the linked complaint. Assigned Department, Status and Reported At are disabled on the issue form when it loads, and issue validation blocks a direct edit of the status, directing users to progress updates instead. These controls guide the demonstrated workflow; they are not an advanced security or immutable audit-trail implementation.

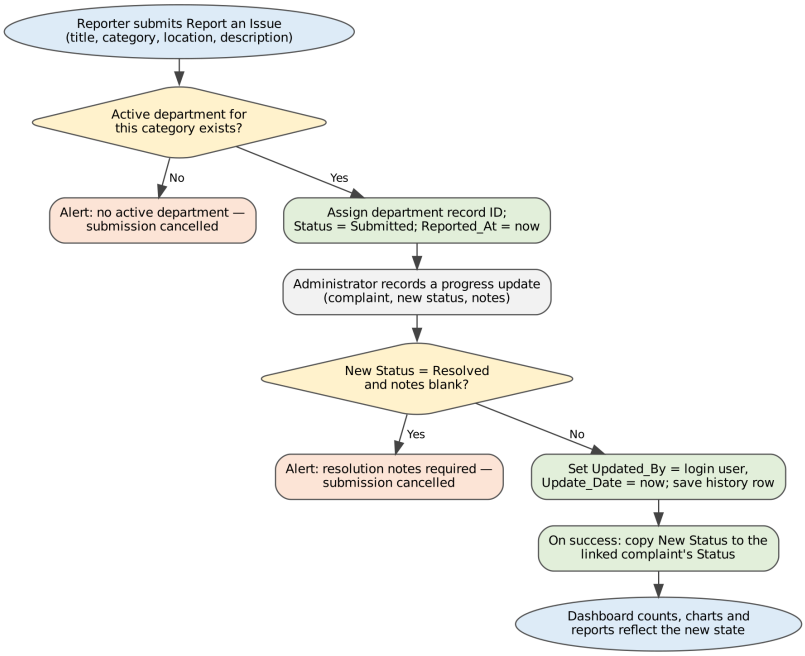


Figure 4.3: Workflow / Process Flow of a Complaint

4.5 User Interface and Navigation Design

The application menu has four sections (Figure 4.4). Each of the three form sections contains its entry form and its list report, and the fourth section opens the Admin Dashboard. The dashboard layout places four count cards across the top (Total, Pending, In Progress, Resolved), the status pie chart and department column chart in the middle, and the All Complaints report below; the implemented page is shown in Figure 5.6.

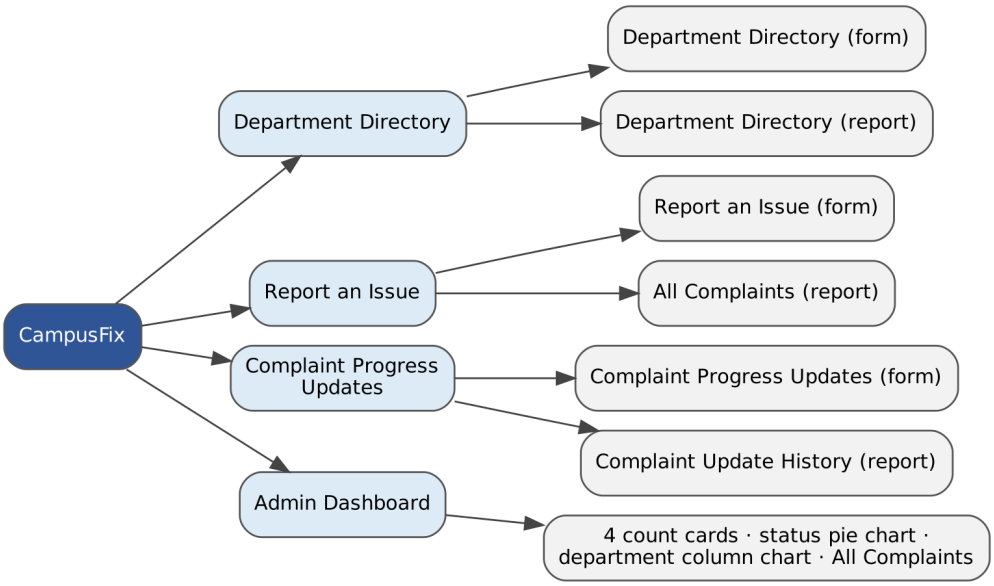


Figure 4.4: Navigation Structure of CampusFix

4.6 Access Control Design

The intended access model is shown in Table 4.5. In this version both paths were operated through the owner account and the matrix was **not** configured as separate Creator roles; it is recorded here as the design for a future multi-user rollout.

Table 4.5: Role–Permission Matrix (Design – Not Yet Configured)

Component	Administrator	Reporter
Form: Department Directory	A E V D	—
Form: Report an Issue	A E V	A V (own)
Form: Complaint Progress Updates	A E V	—
Report: All Complaints	V	V (own)
Report: Complaint Update History	V	—
Page: Admin Dashboard	V	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in the owner's Zoho Creator workspace with the name **CampusFix** and link name `campusfix`. The menu was organised into the four sections of Figure 4.4: Department Directory, Report an Issue, Complaint Progress Updates and Admin Dashboard. Each form section holds the form and its list report, so a user can enter a record and immediately see it listed. The figures in this chapter are screenshots of the live application.

5.2 Forms and Data Capture

Report an Issue captures title, category, location and description, with an optional photo field. Assigned Department, Status and Reported At are controlled by the workflows and are disabled when the form loads. Figure 5.1 shows a new issue before submission; the department and reporting time are filled at save.

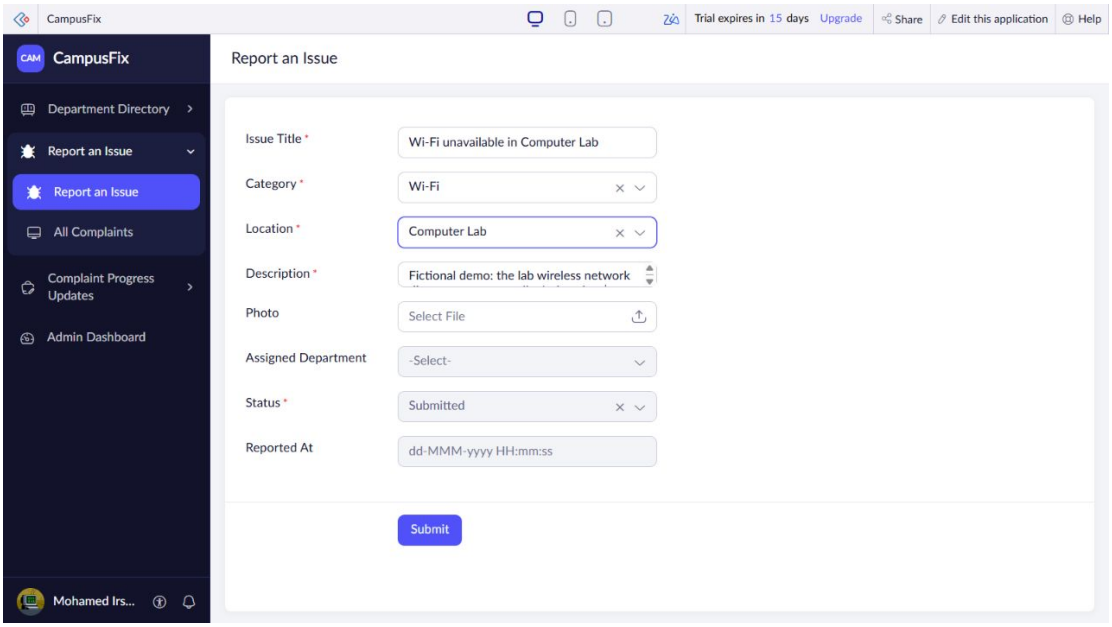


Figure 5.1: Report an Issue Form before Submission

Complaint Progress Updates selects an existing issue through a lookup, records a new status and accepts action notes. Figure 5.2 shows a prepared resolution update; Updated By and Update Date are populated during validation, so their empty appearance before submission is expected.

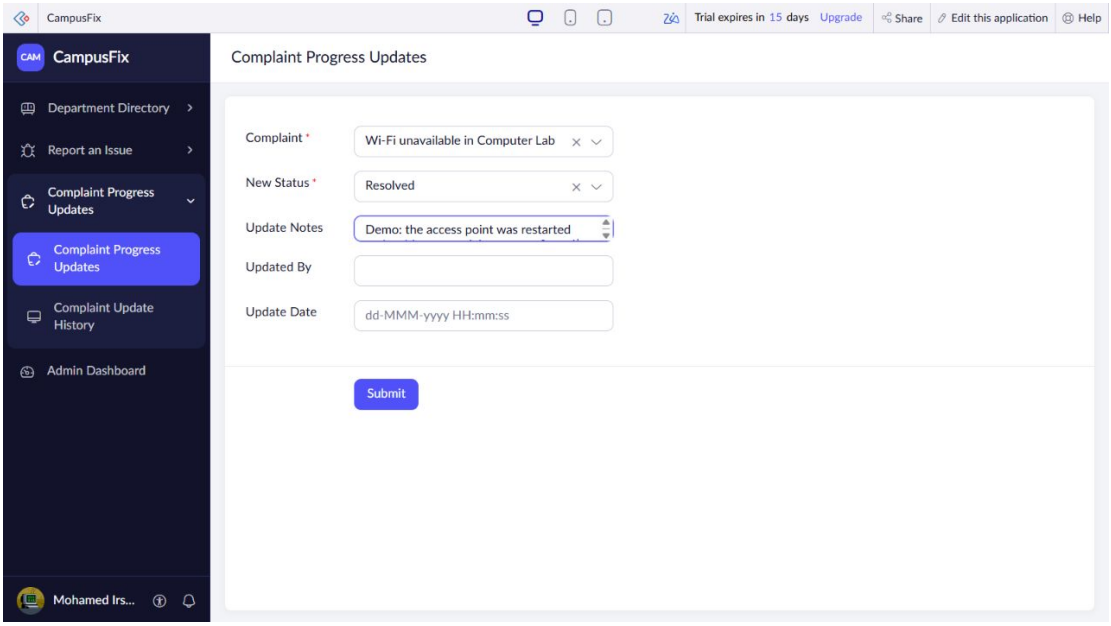


Figure 5.2: Complaint Progress Updates Form with Resolution Notes

Department Directory maintains the responsible team, the category handled, a contact person and e-mail, and the active state. Department Name is unique and Active Status defaults to Active (Figure 5.3). The demonstration holds one active record for each of the five categories; no notification workflow sends mail to the fictional contact addresses.

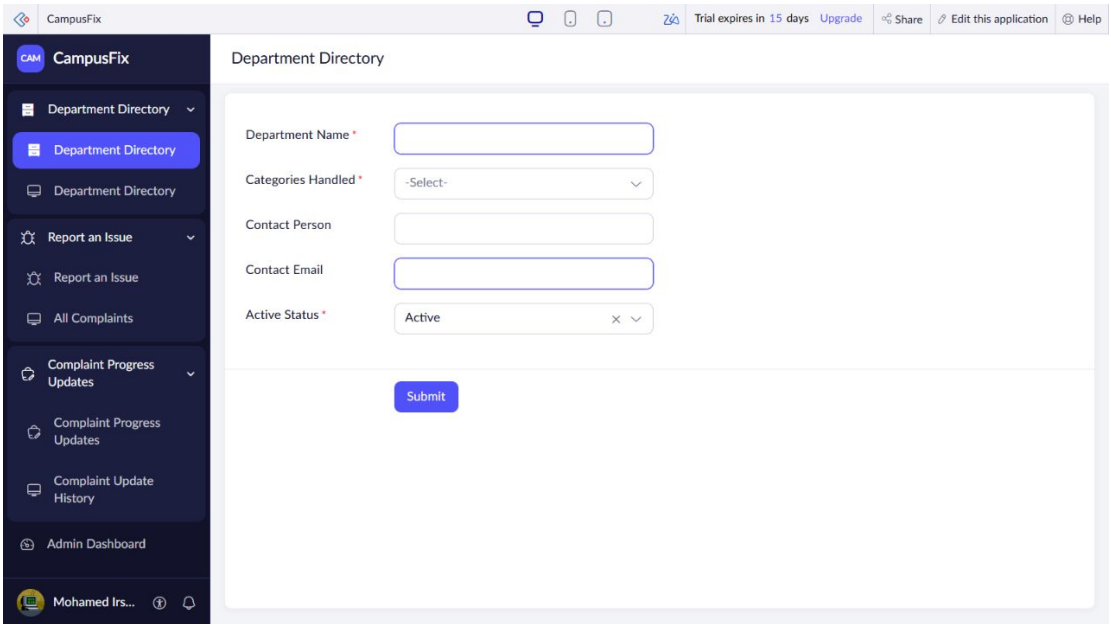


Figure 5.3: Department Directory Entry Form with Active Default

5.3 Reports

Three list reports were created (Table 5.1). The two analytical views are native charts on the dashboard page rather than standalone pivot reports (see Section 5.9).

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Columns / Purpose	Custom Actions	Roles
Department Directory	List	Department Directory	Name, category, contact person, e-mail, active status	—	Owner
All Complaints	List	Report an Issue	Title, category, location, assigned department, status, reported at, description	—	Owner
Complaint Update History	List	Complaint Progress Updates	Complaint, new status, notes, updated by, update date	—	Owner

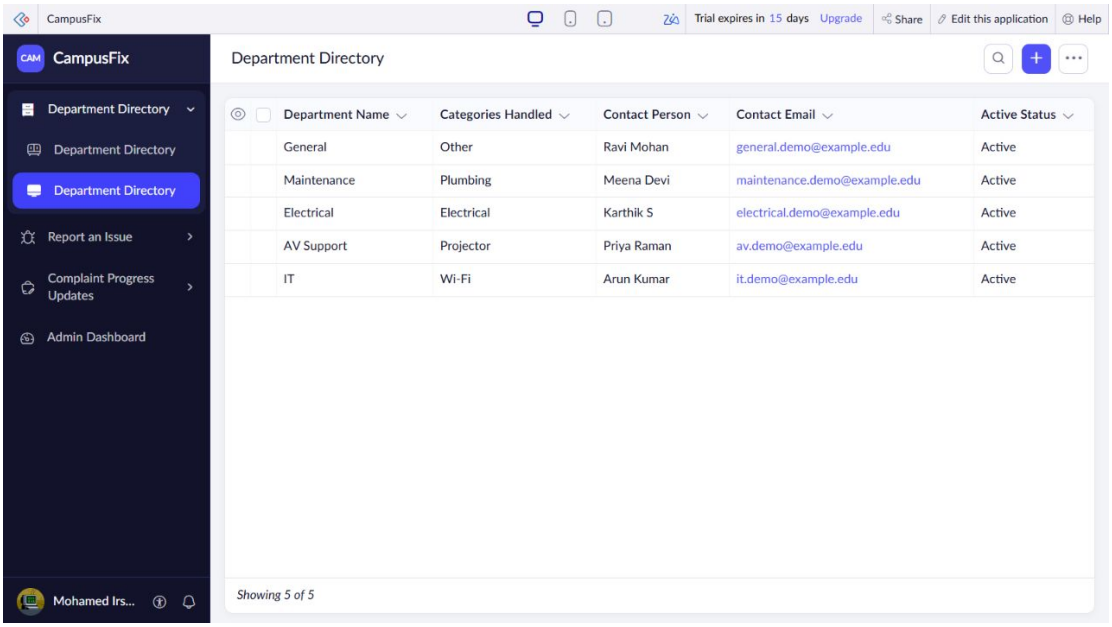


Figure 5.4: Department Directory Report with Five Active Records

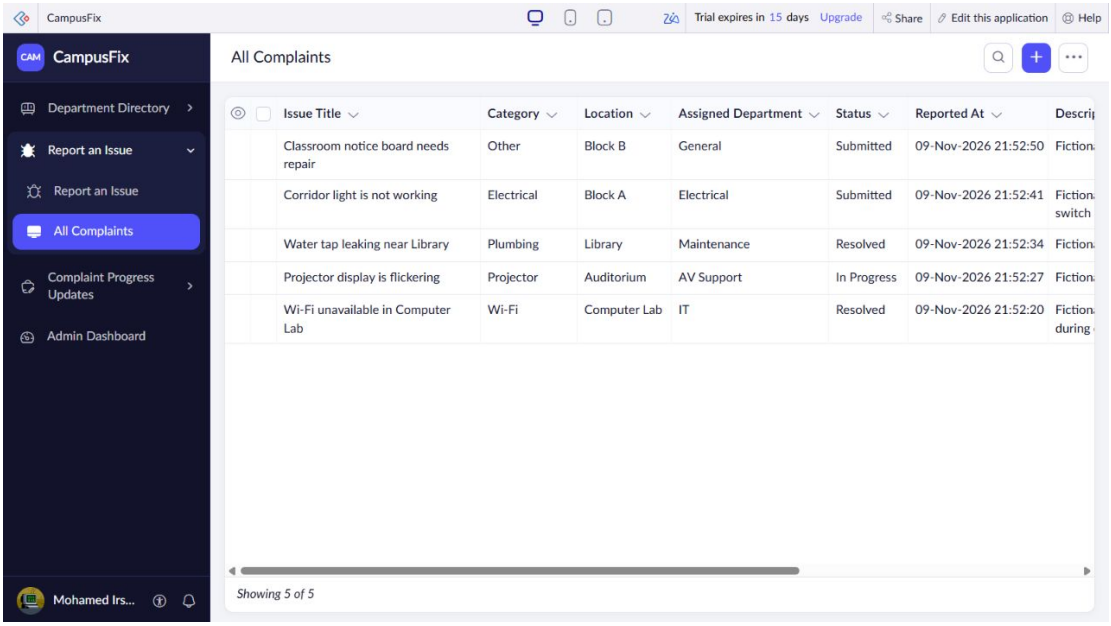


Figure 5.5: All Complaints Report with Assigned Departments and Statuses

5.4 Pages and Dashboards

The **Admin Dashboard** page contains four count cards — Total Complaints, Pending Complaints, In Progress and Resolved (Pending is defined as Submitted) — a pie chart grouping complaints by Status, a column chart grouping them by assigned department, and the embedded All Complaints report for detail (Figure 5.6). Table 5.2 lists the counts verified against the final demonstration data.

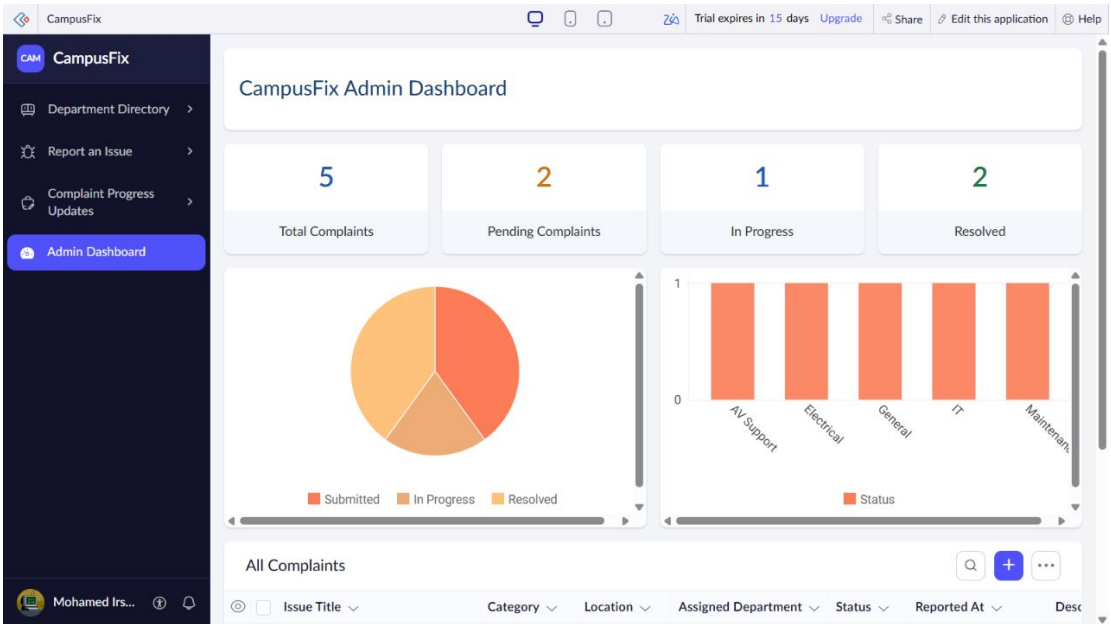


Figure 5.6: Admin Dashboard with Count Cards, Status Chart and Department Workload Chart

Table 5.2: Verified Dashboard Counts

Measure	Value
Total Complaints	5
Pending (Submitted)	2
In Progress	1
Resolved	2

5.5 Deluge Scripting

Four Deluge scripts implement the business rules (Table 5.3); the complete code is in Appendix C. Lookups are always set and read by record ID, which preserves relationships and avoids comparing lookup fields with ordinary department-name text [4].

Table 5.3: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Report_an_Issue → Load of the form	Disable auto-filled fields	disable task
DS-02	Report_an_Issue → Validations on form submission	Assign active department; initialise new issue; guard direct status edit	criteria fetch, count(), if-else, alert, cancel submit, zoho.currenttime
DS-03	Complaint_Progress_Updates → Validations on form submission	Require complaint and resolution notes; stamp updater and time	ifnull, trim, alert, cancel submit, zoho.loginuserid
DS-04	Complaint_Progress_Updates → Successful form submission	Copy new status to the linked complaint	fetch by ID, update by assignment

Code 5.1: Category-based department assignment and initialisation (DS-02)

```
// Script ID : DS-02
// Workflow  : Issue - Assign and Initialize
// Location  : Form Report_an_Issue -> Created or Edited ->
Validations on form submission
// Purpose   : Assign the active department for the category,
initialise a new
//             complaint and block direct status edits
department_record = Department_Directory[Categories_Handled ==
input.Category && Active_Status == "Active"];
if(department_record.count() == 0)
{
  alert "No active department handles this category. Add it to
Department Directory first.";
  cancel submit;
}
input.Assigned_Department = department_record.ID;
if(input.ID == null)
{
  input.Status = "Submitted";
```

```
  input.Reported_At = zoho.currenttime;
}
else
{
  saved_issue = Report_an_Issue[ID == input.ID];
  if(input.Status != saved_issue.Status)
  {
    alert "Record status changes through Complaint Progress Updates to
preserve the history.";
    cancel submit;
  }
}
```

DS-02 fetches the Department Directory record whose category matches the issue and whose status is Active, and stores that record's ID in the Assigned_Department lookup. For a new record (`input.ID == null`) it sets Status to Submitted and Reported At to the current time; for an edit it compares the submitted status with the saved one and cancels the submission if the status was changed directly.

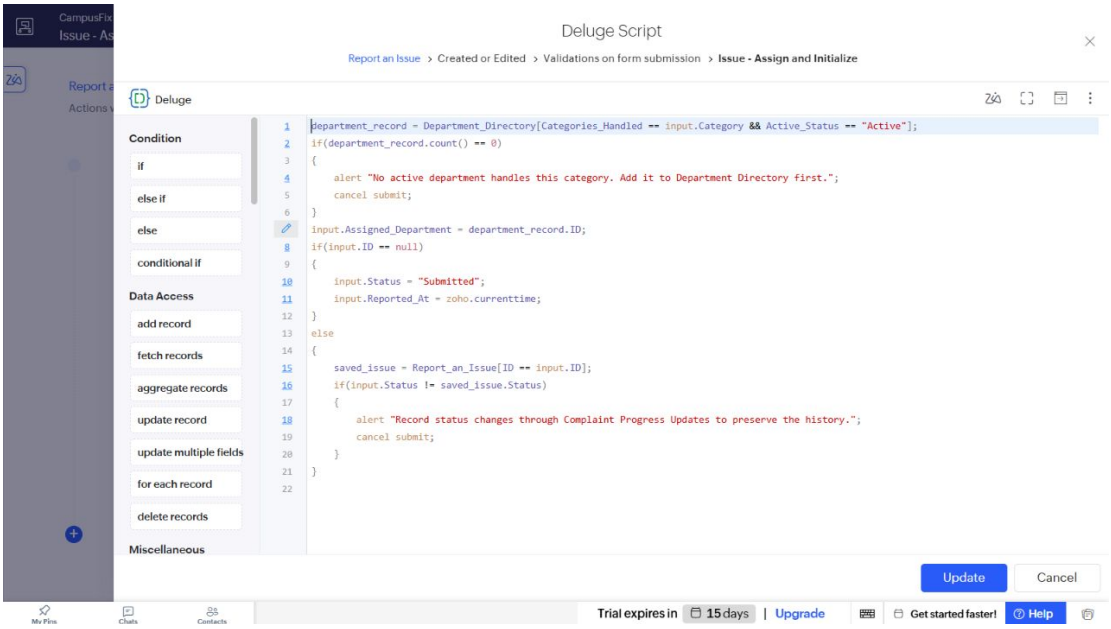


Figure 5.7: Saved Category-Assignment and Initialisation Script in the Deluge Editor

Code 5.2: Resolution-note validation and linked-status synchronisation (DS-03, DS-04)

```
// Script ID : DS-03
// Workflow  : Progress - Validate Notes
// Location  : Form Complaint_Progress_Updates -> Created or Edited ->
Validations on form submission
// Purpose   : Require a complaint and non-blank notes for Resolved;
stamp updater and time
if(input.Complaint == null)
{
  alert "Select an existing complaint.";
  cancel submit;
}
if(input.New_Status == "Resolved" &&
ifnull(input.Update_Notes,"").trim() == "")
```

```
{
  alert "Resolution notes are required before marking a complaint
Resolved.";
  cancel submit;
}
input.Updated_By = zoho.loginuserid;
input.Update_Date = zoho.currenttime;

// Script ID : DS-04
// Workflow  : Progress - Sync Complaint Status
// Location  : Form Complaint_Progress_Updates -> Created or Edited ->
Successful form submission
// Purpose   : Copy New Status to the linked complaint (looked up by
record ID)
complaint_record = Report_an_Issue[ID == input.Complaint];
complaint_record.Status = input.New_Status;
```

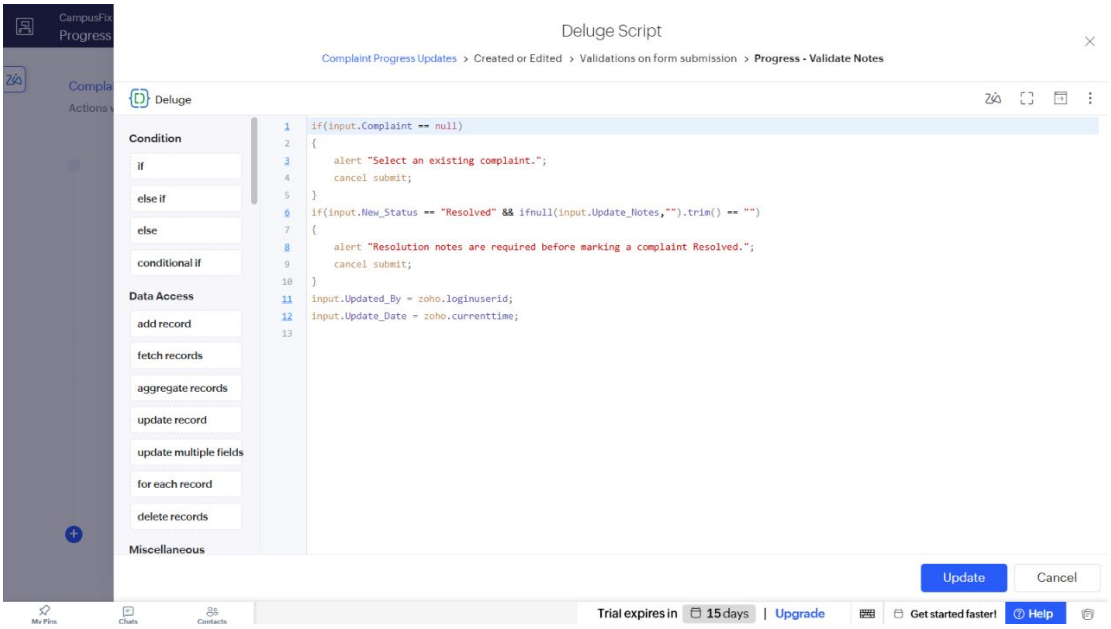


Figure 5.8: Progress-Note Validation Script in the Deluge Editor

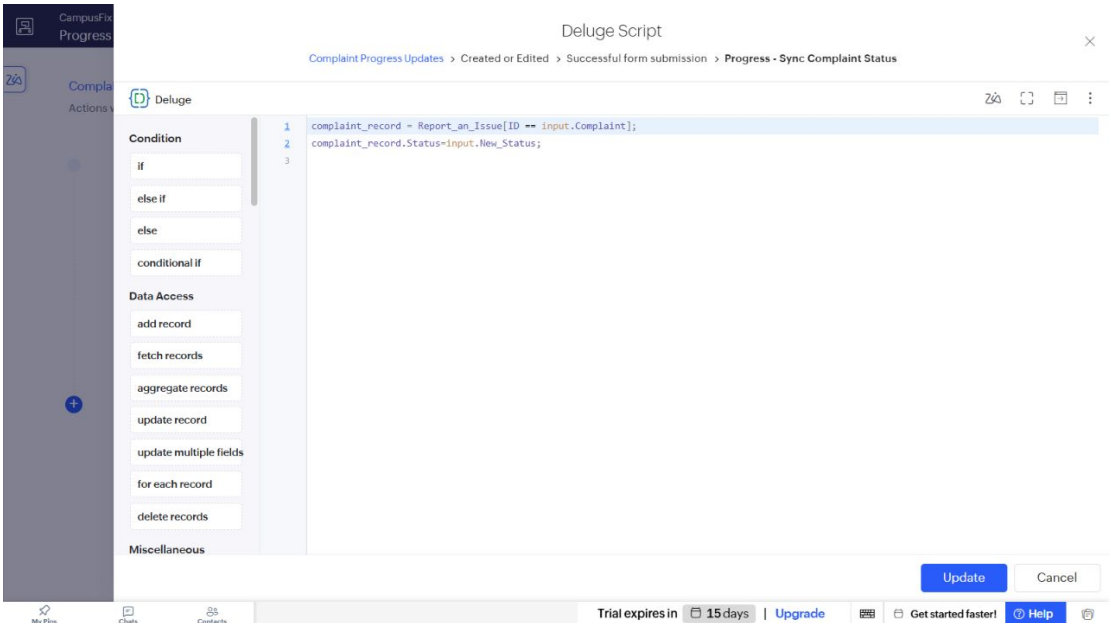


Figure 5.9: Linked-Complaint Status Synchronisation Script

5.6 Workflows and Automation

Table 5.4: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Issue - Automatic Fields	Form Workflow	Report_an_Issue: Load of the form	—	Disable Assigned Department, Status, Reported At	DS-01
Issue - Assign and Initialize	Form Workflow	Report_an_Issue: Validations on form submission	Active mapping exists	Assign department; set Submitted and time; block direct status edit	DS-02
Progress - Validate Notes	Form Workflow	Complaint_Progress _Updates: Validations on form submission	Complaint selected; notes for Resolved	Reject invalid update; set updater and time	DS-03
Progress - Sync Complaint Status	Form Workflow	Complaint_Progress _Updates: Successful form submission	—	Set linked complaint Status	DS-04

All four workflows are enabled (Figure 5.10, Figure 5.11). Scheduled workflows, approval processes, Blueprints and batch workflows were not required for the demonstrated scope and were not used.

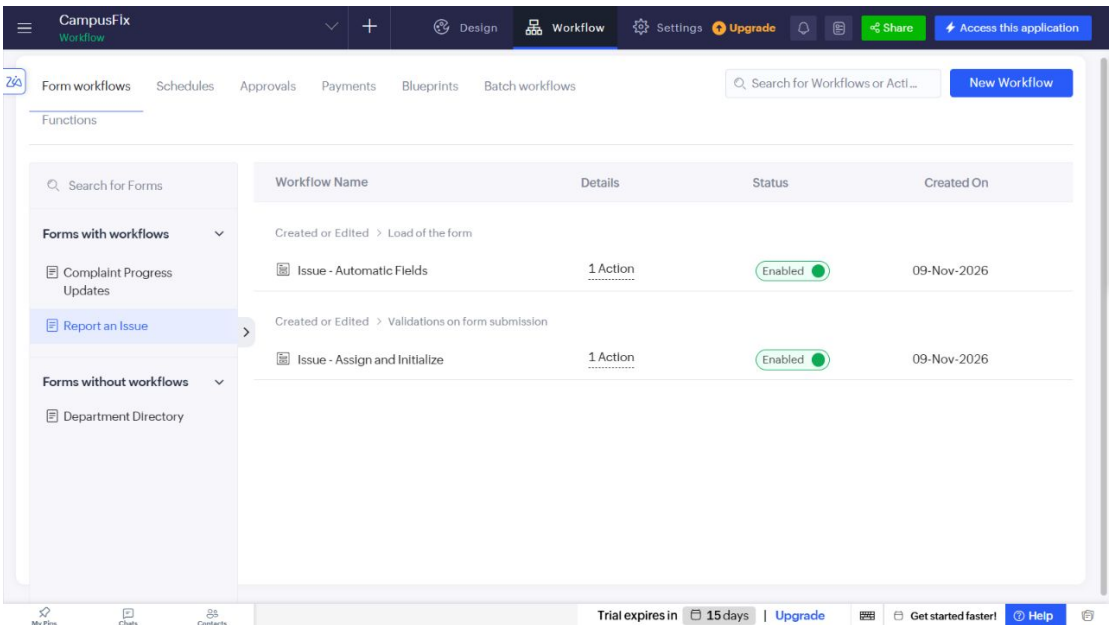


Figure 5.10: Enabled Issue-Form Load and Validation Workflows

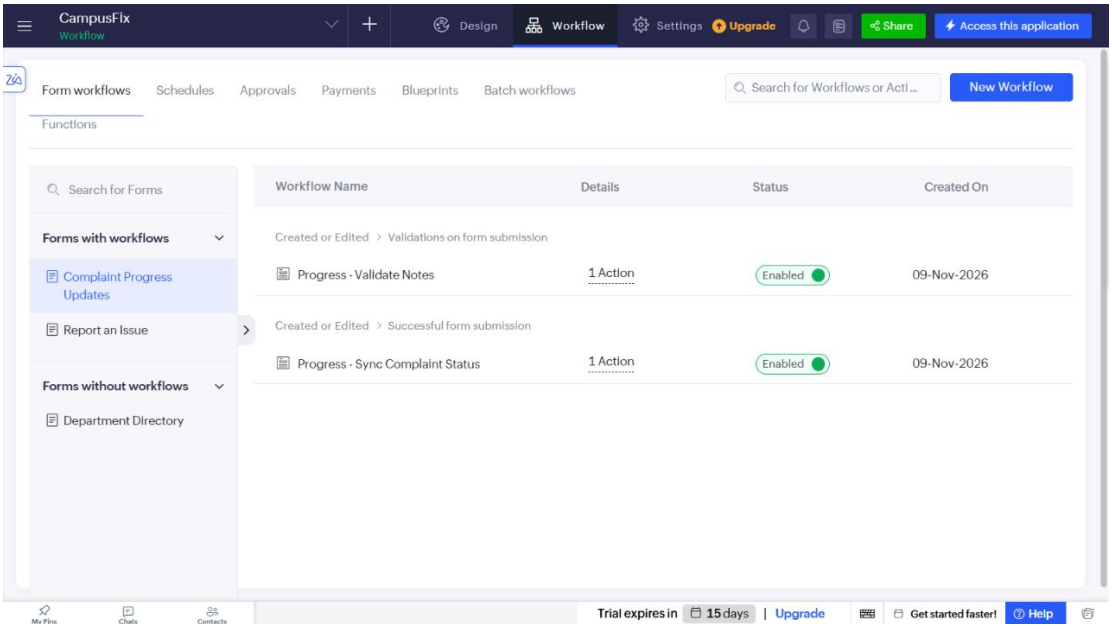


Figure 5.11: Enabled Progress Validation and Synchronisation Workflows

5.7 Integrations

No integration was implemented. The project deliberately keeps all logic inside Creator: routing and status updates work entirely on Creator records, and the demonstration's fictional contact e-mails must not receive messages. E-mail notification to the assigned department (through Deluge `sendmail`) and Creator REST API access are identified as future enhancements in Section 9.2.

5.8 User Management and Role-Based Access

The application was operated through the authorised owner account; separate users, roles and permission profiles were not configured. Disabled fields on the issue form and server-side validation (DS-02, DS-03) guide data entry, and Updated By records the logged-in user ID for every progress update. The access design for a multi-user rollout is given in Table 4.5.

5.9 AI and Analytics Features

Analytics are provided by the dashboard's four count cards and two native page charts: a pie chart of complaints by Status and a column chart of complaints by assigned department. A standalone pivot-chart report was attempted, but its creation path required a Zoho Analytics configuration; that path was cancelled to keep the project free of external integrations, and the native page charts were used instead. Zia AI features were not used in the application.

5.10 Pro-Code Extension

No pro-code extension (Zoho Catalyst functions, JavaScript widgets or external services) was used, because the four Deluge workflows meet every functional requirement. A pro-code component would become worthwhile for features such as a QR-code generator that opens the issue form with the location pre-filled, or a custom visual floor map of complaint locations.

5.11 Mobile and Portal Access

Zoho Creator makes the same application available in its mobile app, but mobile use and a customer portal for students were not configured or tested in this project.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Tests were executed in the live, authenticated owner session on 9 November 2026 using fictional campus issues. **Unit testing** checked each form and each workflow branch: routing for every category, initial values, the blank-notes rejection and status synchronisation. **Integration testing** followed complete paths from issue submission through progress updates to the dashboard counts. **Role-based access testing** and **user acceptance testing with independent users** were not carried out, because the demonstration uses a single owner account. Each PASS below is an observed result. Five valid issues and four valid progress rows were retained; the blank-notes attempt was rejected and did not create a record.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-03	Route Wi-Fi complaint	Submit Wi-Fi complaint	IT assigned	IT assigned	Pass
TC-02	FR-03	Route Projector complaint	Submit Projector complaint	AV Support assigned	AV Support assigned	Pass
TC-03	FR-03	Route Plumbing complaint	Submit Plumbing complaint	Maintenance assigned	Maintenance assigned	Pass
TC-04	FR-03	Initial values	Create five new complaints	Status Submitted; reporting time set	Submitted and reporting time set	Pass
TC-05	FR-06, FR-08	Progress updates and sync	Update Wi-Fi complaint to In Progress, then Resolved	Status changes; two history rows retained	Status changed; two history rows retained	Pass
TC-06	FR-07	Resolution-note validation	Resolve with blank notes, then with valid notes	Blank rejected; valid notes saved	Blank rejected; valid notes saved	Pass
TC-07	FR-03	Route Electrical and Other	Submit Electrical and Other complaints	Electrical and General assigned	Electrical and General assigned	Pass
TC-08	FR-06	History retention	Inspect Complaint Update History	Updater and time stored; four rows	Updater/time stored; four rows retained	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-09	FR-09	Dashboard totals	Open Admin Dashboard	Counts 5/2/1/2; both charts rendered	5/2/1/2 counts; both charts rendered	Pass

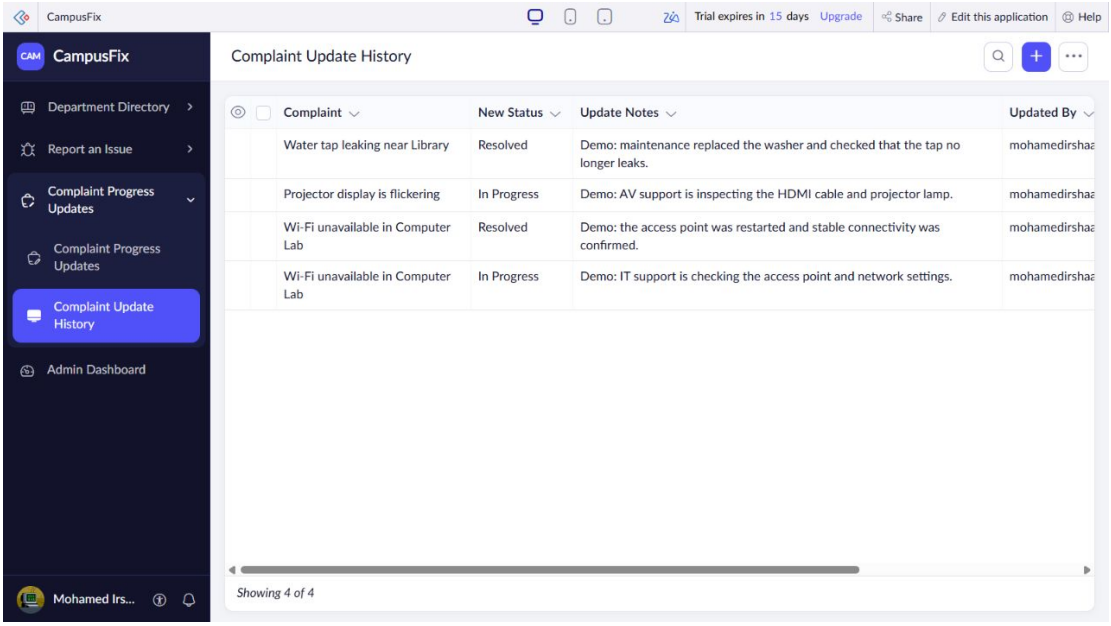


Figure 6.1: Complaint Update History with Four Retained Progress Records

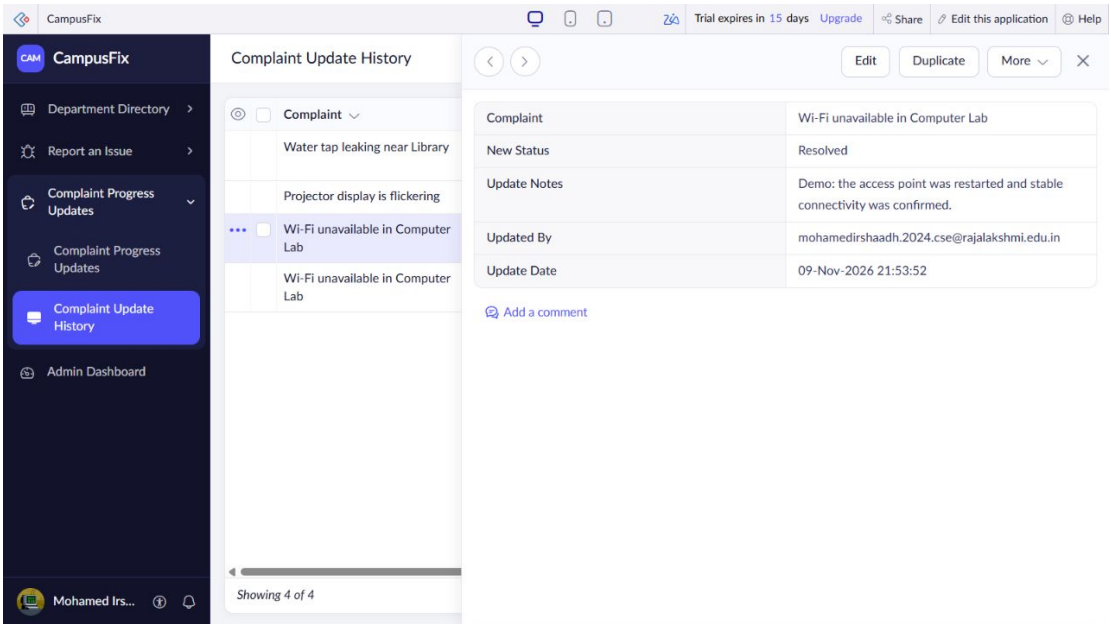


Figure 6.2: Saved Resolution Update with Updater and Timestamp

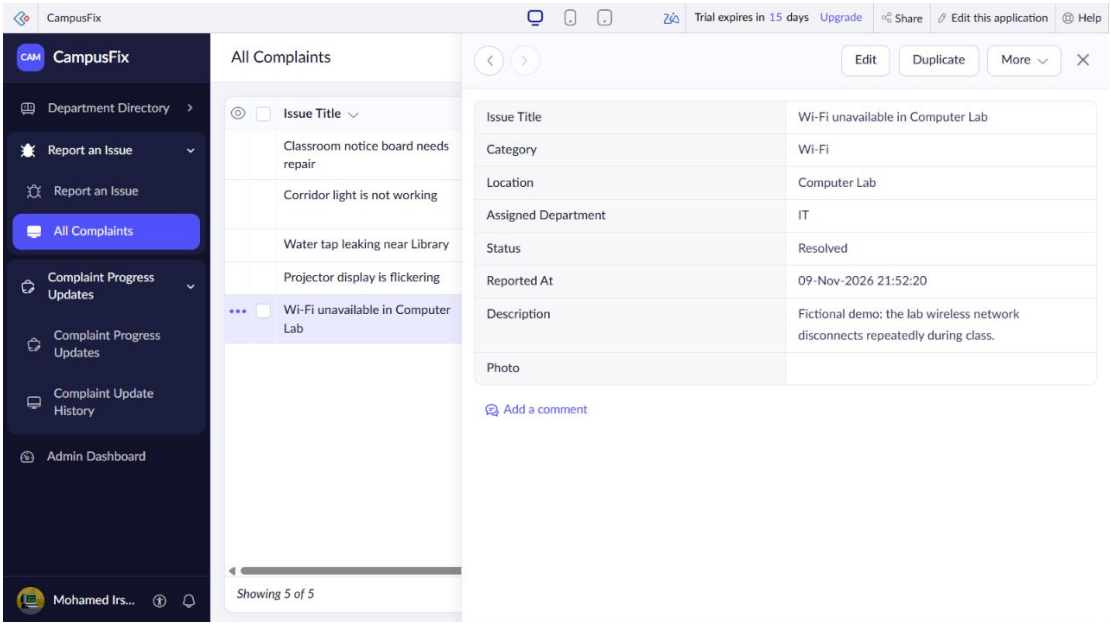


Figure 6.3: Saved Wi-Fi Complaint Assigned to IT and Marked Resolved

6.3 Debugging Deluge Scripts

The scripts were developed in the Creator Deluge editor, which validates syntax and offers field references by link name; link names in the scripts were checked against the saved form definitions. The main validation was debugged by submitting the negative case deliberately: a Resolved update with empty notes. The check uses `ifnull(input.Update_Notes,"").trim() == ""`, so empty and whitespace-only notes are both rejected. The live retest in Figure 6.4 shows the submission being rejected with the validation message, and no extra progress record was created. The positive path — the same update with valid notes — saved correctly (Figure 6.2).

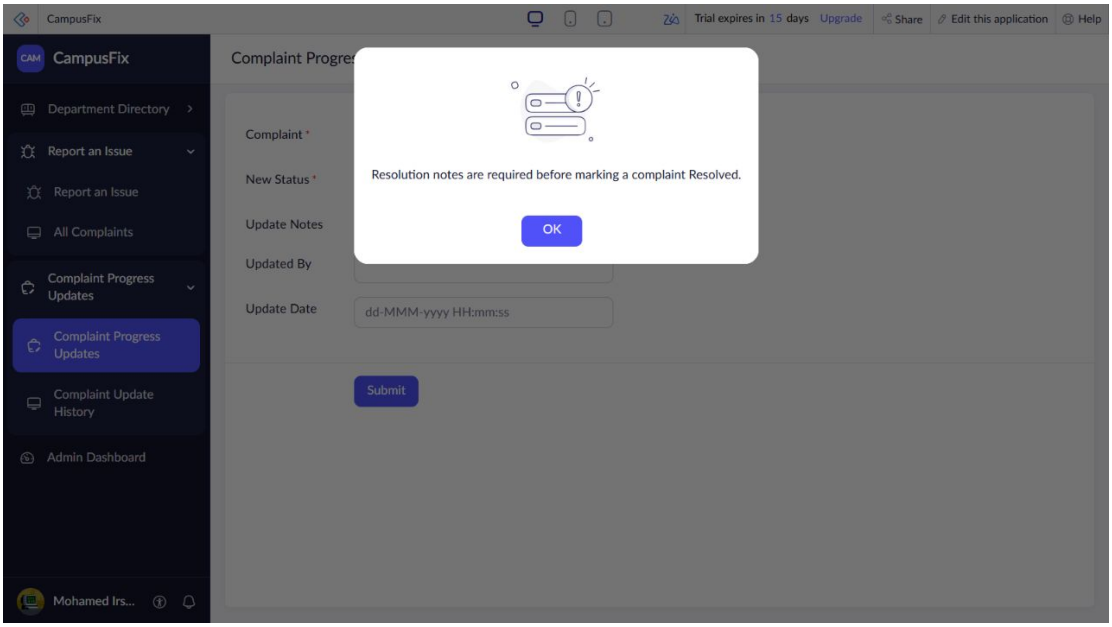


Figure 6.4: Live Rejection of a Resolved Update without Notes

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Standalone pivot-chart report could not be created within Creator alone	Pivot-chart creation path requires Zoho Analytics configuration	Cancelled that path; used native page charts on the Admin Dashboard	Closed

6.5 User Acceptance Testing

User acceptance testing with independent users was not conducted; all tests were run by the developer in the owner session. Independent-account testing is listed as the first step of a wider rollout (Section 7.4).

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
—	—	Not conducted in this version	Planned before any campus rollout

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

CampusFix runs in the owner's Zoho Creator workspace on a trial plan and is accessed through ordinary hosted Creator access at https://creatorapp.zoho.in/mohamedirshaadh.2024.cse1405/campusfix/. No separate Development, Stage and Production promotion, external portal or mobile acceptance process was performed. For a campus rollout, the application would be developed and tested in the Development environment, verified by independent users in Stage, and then published to Production with version notes, keeping the previous version available for rollback.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Status
1	All test cases executed	Done (TC-01 to TC-09)
2	Sample/test data removed or isolated	Not done – fictional demo data retained for evaluation
3	Roles, permissions and users configured	Not done – owner account only
4	Connections authorised	Not applicable – no integrations
5	Workflows enabled	Done – four form workflows enabled
6	Application URL verified	Done
7	Version / release notes recorded	Not done

7.2 Performance Optimisation

No performance measurement was made, and the data set is small (five departments, five complaints, four updates). The design nevertheless avoids common inefficiencies, as listed in Table 7.2; the effects are described qualitatively and are not measured improvements.

Table 7.2: Performance-Related Design Choices

Design Choice	Component	Alternative Avoided	Effect
Single criteria fetch on category and active status	DS-02	Fetching all departments and looping	One query per submission
Lookup stored as record ID	DS-02, DS-04	Matching by department name or issue title text	Direct record access; no ambiguity
Only the linked complaint updated	DS-04	Re-scanning all complaints	One record update per progress entry
Native page charts	Admin Dashboard	External analytics integration	No external calls when the page loads

7.3 Security Features

Table 7.3: Security Controls and Remaining Boundaries

Control	Where Applied	Current State / Threat Mitigated
Zoho account login	Application access	Only the authorised owner account can open the application
Disabled system fields	Report an Issue (DS-01)	Prevents manual entry of department, status and time
Server-side validation	DS-02, DS-03	Rejects unmapped categories, direct status edits and Resolved without notes
Lookups by record ID	Assigned_Department, Complaint	Prevents mismatched or spoofed text references
Updater identity captured	Updated_By = zoho.loginuserid	Each update attributable to a user
Not implemented	Roles / profiles, encryption, OAuth connections	Separate-user access, load limits and photo-upload handling remain unverified

7.4 Governance and Administration

Administration is kept simple. The five category mappings in Department Directory should remain active, and competing mappings for the same category should not be created without a routing policy. Workflow names describe their form and purpose (for example *Issue - Assign and Initialize*, *Progress - Sync Complaint Status*), and field link names match their labels. Source definitions and test evidence should be retained before changing a workflow, and the relevant tests rerun afterwards. Progress records are ordinary editable Creator records rather than an immutable audit log; enabling Creator's

audit trail, adding user permissions, retention rules, backup checks and independent acceptance tests would be required before a wider campus rollout. No password or credential is included in this report.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The working application was demonstrated with five fictional complaints. **Step 1:** each complaint was submitted through Report an Issue, and all five categories were routed to their intended active departments with status Submitted and a reporting timestamp (Figure 5.5). **Step 2:** progress updates were recorded — the Wi-Fi issue received In Progress and then Resolved updates, the projector issue reached In Progress and the plumbing issue reached Resolved — and each update changed the linked complaint status while being kept as a separate history row (Figure 6.1). **Step 3:** the Admin Dashboard showed the final counts and both charts (Figure 5.6). Table 8.1 summarises the final state.

Table 8.1: Final Demonstration State (Fictional Records)

Category	Department	Final Status
Wi-Fi	IT	Resolved
Projector	AV Support	In Progress
Electrical	Electrical	Submitted
Plumbing	Maintenance	Resolved
Other	General	Submitted

8.2 Achievement of Objectives

Table 8.2: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
1. Three linked forms	Sections 4.2, 4.3, 5.2	Three forms, two lookups; Figures in Section 5.2
2. Automatic department assignment	DS-02	TC-01 to TC-03, TC-07
3. Lifecycle with separate history rows	DS-03, DS-04	TC-05, TC-08
4. Resolution notes required	DS-03	TC-06; live rejection screenshot
5. Reports and Admin Dashboard	Sections 5.3, 5.4	TC-09; counts 5/2/1/2
6. Documented functional testing	Chapter 6	Nine test cases passed on 9 November 2026

8.3 Comparison with the Existing System

Table 8.3: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Recording a complaint	Verbal or message, no record	Structured form with timestamp	Every complaint recorded
Choosing the department	Receiver decides and forwards	Assigned from active category mapping	Consistent routing
Knowing the status	Repeated follow-up	Current status on the complaint	Visible at any time
History of actions	Not kept	Separate progress rows with notes, updater, time	Traceable
Administrative overview	Compiled manually	Dashboard counts and charts	Immediate

These are functional improvements demonstrated with fictional data. No measured reduction in repair time, administrative cost or complaint volume is claimed; such outcomes would need real adoption and a defined comparison period.

8.4 Limitations of the Low-Code Approach

The project met some platform constraints and has its own limits. Creating a standalone pivot chart required Zoho Analytics, so analysis is limited to the native page charts. Lookups do not synchronise related fields automatically, so a Deluge success workflow was needed to copy the status to the complaint. Progress records are ordinary editable records, not an immutable audit log, and the prototype does not enforce a one-way state-transition model or resolve simultaneous updates. The application has a small owner-session data set and was not tested with independent users. Finally, the forms and Deluge scripts are specific to Zoho; the data can be exported, but the logic would have to be rewritten on another platform [7].

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of CampusFix is **no-code configuration**: the three forms, field types and defaults, the lookups, the list reports, the dashboard page with its count cards and charts, and the menu. The **low-code** layer is four short Deluge scripts that hold the business rules — category routing, initial values, resolution-note validation and linked-status synchronisation — which configuration alone could not express. **Pro-code** was not

needed at this scale; it would become appropriate for features such as QR-code entry points with pre-filled locations or a custom campus map view. The project showed that the decision to move along the spectrum should follow concrete requirements rather than the wish to write more code [8].

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

CampusFix meets its central objective of a simple, coherent Zoho Creator issue tracker for the campus. Three linked forms, four form workflows, three list reports and one dashboard page provide structured complaint records, automatic routing to the responsible department, controlled status management with a separate progress history, and an administrative overview. All nine functional test scenarios passed in the live owner-session demonstration.

The implementation is deliberately modest and keeps a clear distinction between fictional demonstration records and real maintenance operations. It shows how a low-code platform and a small amount of Deluge can replace informal complaint handling with a traceable, centrally visible process.

9.2 Future Enhancements

- **Role-based access:** separate reporter, department-technician and administrator roles, with reporters seeing only their own complaints, after independent-account testing.
- **Notifications:** e-mail to the assigned department on submission and to the reporter on resolution.
- **Priority and escalation:** priority levels with target resolution times and reminders for overdue complaints.
- **QR codes in classrooms** that open the issue form with the location pre-filled.
- **Feedback and analytics:** reporter rating after resolution, and average resolution time by department.
- **Controlled transitions:** a Blueprint enforcing the Submitted → In Progress → Resolved order and protecting historical updates.

9.3 Learning Outcomes and Career Relevance

The project applied relational data modelling, Creator form configuration, event-based automation with load, validation and success workflows, Deluge record retrieval by criteria and by ID, input validation and dashboard grouping. It also developed the practice of recording only observed test results and documenting a platform constraint

— the pivot-chart dependency on Zoho Analytics — rather than concealing it. These skills are directly relevant to roles such as low-code developer, business-process automation specialist and solutions consultant, and form a foundation for moving from citizen development towards professional application development.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Understanding Form Builder," Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/forms/the-concept-of-forms/articles/form-builder (Accessed: Nov. 2026).

[3] Zoho Corporation, "Creating and Managing Form Workflows," Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-form-workflows/articles/create-new-form-workflow (Accessed: Nov. 2026).

[4] Zoho Corporation, "Criteria to Fetch Records," Deluge Help. [Online]. Available: https://www.zoho.com/deluge/help/criteria.html (Accessed: Nov. 2026).

[5] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2023.

[6] R. Sanchis, Ó. García-Perales, F. Fraile, and R. Poler, "Low-code as enabler of digital transformation in manufacturing industry," *Applied Sciences*, vol. 10, no. 1, p. 12, 2020.

[7] A. C. Bock and U. Frank, "Low-code platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[8] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.

[9] AXELOS, *ITIL Foundation: ITIL 4 Edition*. London, U.K.: TSO, 2019.

[10] ServiceNow, Inc., "Incident Management," ServiceNow Product Documentation. [Online]. Available: https://docs.servicenow.com (Accessed: Nov. 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	CampusFix – 4 menu sections	5.1	Figure 4.4
II	Forms with ≥ 8 distinct field types	Y	Single Line, Multi Line, Drop Down, Email, File Upload, Lookup, Date-Time (7 types)	4.3, 5.2	Table 4.3
II	Lookup field (relationship between forms)	Y	Assigned_Department, Complaint	4.2	Figure 4.2
II	Subform (parent–child data)	N	—	—	—
II	Field validations / layout rules	Y	Mandatory fields, unique name, DS-01 to DS-03	5.2, 5.5	Code 5.1
II	List report with filter, grouping, custom action	Y (list only)	3 list reports	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	N	—	—	—
II	Pivot table / chart report	Partial	2 native page charts (pivot needs Zoho Analytics)	5.4, 5.9	Figure 5.6
II	Page / dashboard built with UI Builder	Y	Admin Dashboard	5.4	Figure 5.6
II	Deluge – form events (validate / success / field actions)	Y	DS-01 to DS-04	5.5	Code 5.1, 5.2
II	Deluge – functions, fetch, loops, collections	Partial	Criteria fetch and update by ID (no functions)	5.5	Table 5.3
II	Email / in-app notifications	N	—	9.2	—
II	Form workflow	Y	4 form workflows	5.6	Table 5.4
II	Scheduled workflow	N	—	—	—
II	Approval process	N	—	—	—
II	Blueprint	N	—	9.2	—
II	Batch / report workflow	N	—	—	—
II	Testing & debugging evidence	Y	9 test cases, live rejection	6	Table 6.1, Figure 6.4
III	Integration with a Zoho service	N	—	5.7	—
III	Connection / REST API call (invokeurl)	N	—	—	—
III	Webhook or Creator REST API	N	—	—	—

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Users, roles and permission profiles	N	Design only	4.6, 5.8	Table 4.5
III	Record-level access criteria / customer portal	N	—	—	—
III	Zia / AI feature	N	—	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Native page charts and count cards	5.4	Figure 5.6
IV	Performance optimisation applied & measured	Partial	Design choices; not measured	7.2	Table 7.2
IV	Deployment across environments	N	Single owner workspace	7.1	Table 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	Login, disabled fields, server-side validation	7.3	Table 7.3
IV	Governance (audit trail, administration)	Partial	Naming, mapping policy; audit trail not enabled	7.4	—
V	Pro-code extension (Catalyst / widgets / custom functions)	N	—	5.10	—
V	Low-code limitations & spectrum analysis	Y	—	8.4, 8.5	—
—	Mobile app / portal access	N	Not tested	5.11	—

Total features implemented: 10 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7, 5.8 (design only)	1	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	1	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 2.5, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 DS-01 – Report_an_Issue – Load of the form (Issue - Automatic Fields)

```
// Script ID : DS-01
// Workflow  : Issue - Automatic Fields
// Location  : Form Report_an_Issue -> Created or Edited -> Load of
the form
// Purpose   : Prevent manual entry of values supplied by the
validation workflow
disable Assigned_Department;
disable Status;
disable Reported_At;
```

C.2 DS-02 – Report_an_Issue – Validations on form submission (Issue - Assign and Initialize)

```
// Script ID : DS-02
// Workflow  : Issue - Assign and Initialize
// Location  : Form Report_an_Issue -> Created or Edited ->
Validations on form submission
// Purpose   : Assign the active department for the category,
initialise a new
//             complaint and block direct status edits
department_record = Department_Directory[Categories_Handled ==
input.Category && Active_Status == "Active"];
if(department_record.count() == 0)
{
  alert "No active department handles this category. Add it to
Department Directory first.";
  cancel submit;
}
input.Assigned_Department = department_record.ID;
if(input.ID == null)
{
  input.Status = "Submitted";
  input.Reported_At = zoho.currenttime;
}
else
{
  saved_issue = Report_an_Issue[ID == input.ID];
  if(input.Status != saved_issue.Status)
  {
    alert "Record status changes through Complaint Progress Updates to
preserve the history.";
    cancel submit;
  }
}
```

C.3 DS-03 – Complaint_Progress_Updates – Validations on form submission (Progress - Validate Notes)

```
// Script ID : DS-03
// Workflow  : Progress - Validate Notes
// Location  : Form Complaint_Progress_Updates -> Created or Edited ->
Validations on form submission
// Purpose   : Require a complaint and non-blank notes for Resolved;
stamp updater and time
```

```
if(input.Complaint == null)
{
  alert "Select an existing complaint.";
  cancel submit;
}
if(input.New_Status == "Resolved" &&
ifnull(input.Update_Notes,"").trim() == "")
{
  alert "Resolution notes are required before marking a complaint
Resolved.";
  cancel submit;
}
input.Updated_By = zoho.loginuserid;
input.Update_Date = zoho.currenttime;
```

C.4 DS-04 – Complaint_Progress_Updates – Successful form submission (Progress - Sync Complaint Status)

```
// Script ID : DS-04
// Workflow  : Progress - Sync Complaint Status
// Location  : Form Complaint_Progress_Updates -> Created or Edited ->
Successful form submission
// Purpose   : Copy New Status to the linked complaint (looked up by
record ID)
complaint_record = Report_an_Issue[ID == input.Complaint];
complaint_record.Status = input.New_Status;
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open https://creatorapp.zoho.in/mohamedirshaadh.2024.cse1405/campusfix/ in Chrome, Edge or Firefox and sign in with an authorised Zoho account. No password is provided in this report.

D.2 Reporting an Issue

- Open **Report an Issue → Report an Issue**. Enter a clear title and choose the category and location; describe the problem.
- Attach a photo only if appropriate (this optional path was not part of the recorded tests).
- Click **Submit**. Open **All Complaints** to confirm the assigned department, the Submitted status and the reporting time.

D.3 Recording Progress (Administrator)

- Open **Complaint Progress Updates → Complaint Progress Updates** and select the existing complaint.
- Choose **New Status** and enter a useful action note. For **Resolved**, notes are mandatory.
- Click **Submit**, then check both the complaint's status in All Complaints and the new row in **Complaint Update History**. Record a separate update for each later action so earlier rows remain visible.

D.4 Reviewing and Maintaining

- Open **Admin Dashboard** for totals, status distribution, department workload and complaint records. *Pending* means Submitted.
- Keep exactly one active **Department Directory** record for each category.
- After changing a mapping or a workflow, rerun the relevant tests in Chapter 6.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	MOHAMED IRSHAADH	Register No.	240701323
Project Title	CampusFix – Smart Campus Issue Tracker	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	MOHAMED IRSHAADH	Register No.	240701323
Department / Section	Computer Science and Engineering /	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____